LAPORAN PRAKTIKUM

Struktur Data

MODUL 6

DOUBLY LINKED LIST (BAGIAN PERTAMA)



Disusun oleh:

Farel Tri Julian

109082500163

S1IF-13-02

PROGRAM STUDI S1 INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

TAHUN 2026

LATIHAN SOAL MODUL

Link Repo Github Modul 6 : https://github.com/FarelTriJulian/Struktur-Data/tree/main/Modul%206

1. **doublylist.h**

   Source Code

```
#ifndef DOUBLYLIST_H
#define DOUBLYLIST_H

#include <iostream>
#include <string>
using namespace std;

struct kendaraan {
    string nopol, warna;
    int thnBuat;
};

typedef kendaraan infotype;
typedef struct ElmList *address;

struct ElmList {
    infotype info;
    address next, prev;
};

struct List {
    address First, Last;
};

void CreateList(List &L);
address alokasi(infotype x);
void dealokasi(address &P);
void printInfo(List L);
void insertLast(List &L, address P);
address findElm(List L, infotype x);
void deleteFirst(List &L, address &P);
void deleteLast(List &L, address &P);
void deleteAfter(address Prec, address &P);

#endif
```

Deskripsi program

Program file header ini merupakan kerangka dasar Doubly Linked List untuk modul praktikum kendaraan versi lain di mana tipe data utamanya menggunakan struct kendaraan yang di typedef sebagai infotype, lengkap dengan definisi node, struktur list, serta daftar purwarupa fungsi dasar, insert, dan delete yang siap dirinci kodenya di file.cpp.

2. doubylist.cpp

Source Code

```
#include "Doublylist.h"

void CreateList(List &L) {
    L.First = NULL;
    L.Last = NULL;
}

address alokasi(infotype x) {
    address P = new ElmList;
    P->info = x;
    P->next = NULL;
    P->prev = NULL;
    return P;
}

void dealokasi(address &P) {
    delete P;
    P = NULL;
}

void printInfo(List L) {
    address P = L.Last;
    while (P != NULL) {
        cout << "no polisi : " << P->info.nopol << endl;
        cout << "warna     : " << P->info.warna << endl;
        cout << "tahun     : " << P->info.thnBuat << endl;
        P = P->prev;
    }
}

void insertLast(List &L, address P) {
    if (L.First == NULL) {
        L.First = P;
        L.Last = P;
    } else {
        P->prev = L.Last;
```

```
            L.Last->next = P;
            L.Last = P;
        }
    }

    address findElm(List L, infotype x) {
        address P = L.First;
        while (P != NULL) {
            if (P->info.nopol == x.nopol) return P;
            P = P->next;
        }
        return NULL;
    }

    void deleteFirst(List &L, address &P) {
        P = L.First;
        if (L.First == L.Last) {
            L.First = NULL;
            L.Last = NULL;
        } else {
            L.First = P->next;
            L.First->prev = NULL;
            P->next = NULL;
        }
    }

    void deleteLast(List &L, address &P) {
        P = L.Last;
        if (L.First == L.Last) {
            L.First = NULL;
            L.Last = NULL;
        } else {
            L.Last = P->prev;
            L.Last->next = NULL;
            P->prev = NULL;
        }
    }

    void deleteAfter(address Prec, address &P) {
        P = Prec->next;
        Prec->next = P->next;
        if (P->next != NULL) P->next->prev = Prec;
        P->next = NULL;
        P->prev = NULL;
    }
```

Deskripsi program

Program file Doublylist.cpp ini berisi kode rincian dari fungsi dan prosedur untuk versi header sebelumnya—yang mengatur pembuatan list kosong, alokasi serta dealokasi memori, pencetakan data dari belakang ke depan (printInfo), penambahan data di akhir (insertLast), pencarian elemen menggunakan tipe data infotype (findElm), serta berbagai prosedur penghapusan elemen seperti deleteFirst, deleteLast, dan deleteAfter.

3. **main.cpp**

Source Code

```
#include "Doublylist.h"

int main() {
    List L;
    CreateList(L);
    infotype x;
    address P;

    for (int i = 0; i < 4; i++) {
        cout << "masukkan nomor polisi: ";
        cin >> x.nopol;
        cout << "masukkan warna kendaraan: ";
        cin >> x.warna;
        cout << "masukkan tahun kendaraan: ";
        cin >> x.thnBuat;

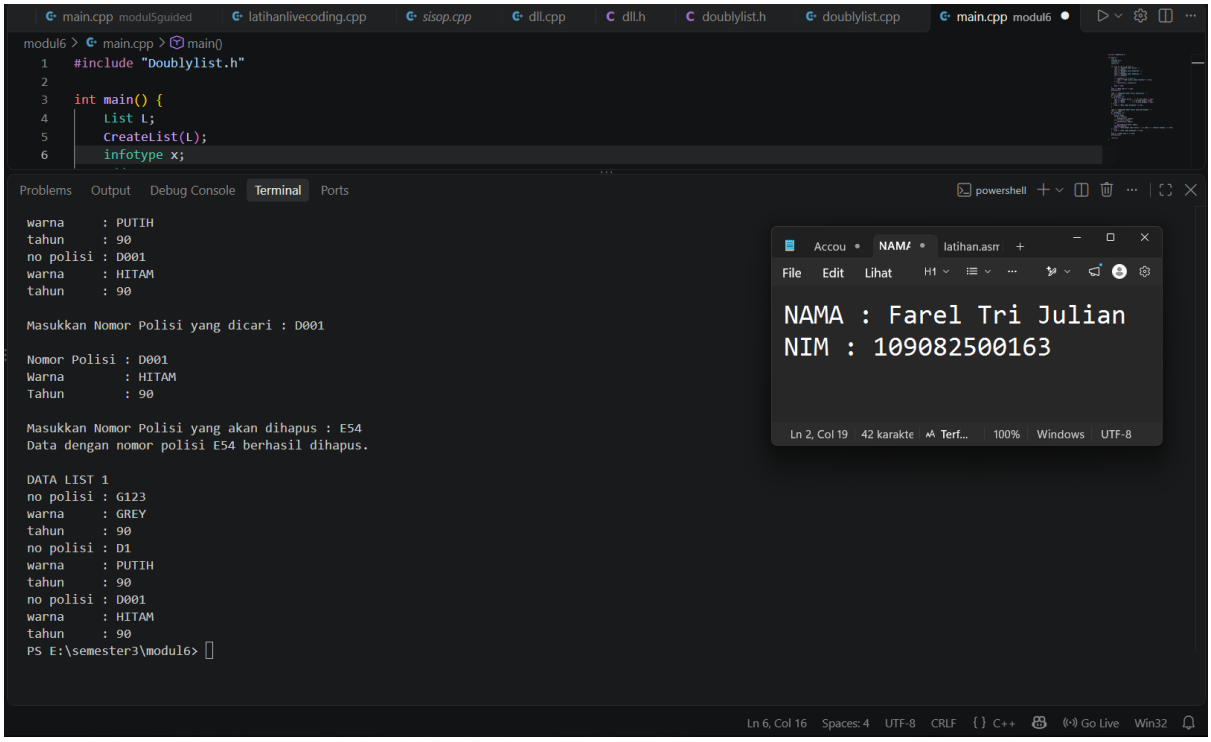
        if (findElm(L, x) != NULL) {
            cout << "nomor polisi sudah terdaftar" << endl;
        } else {
            insertLast(L, alokasi(x));
        }
        cout << endl;
    }
    cout << "DATA LIST 1" << endl;
    printInfo(L);

    cout << "\nMasukkan Nomor Polisi yang dicari : ";
    cin >> x.nopol;
    P = findElm(L, x);
    if (P != NULL) {
        cout << "\nNomor Polisi : " << P->info.nopol << endl;
        cout << "Warna        : " << P->info.warna << endl;
        cout << "Tahun        : " << P->info.thnBuat << endl;
    } else {
        cout << "Data tidak ditemukan" << endl;
    }
```

```
    cout << "\nMasukkan Nomor Polisi yang akan dihapus : ";
    cin >> x.nopol;
    P = findElm(L, x);
    if (P != NULL) {
        address hapus;
        if (P == L.First)
            deleteFirst(L, hapus);
        else if (P == L.Last)
            deleteLast(L, hapus);
        else
            deleteAfter(P->prev, hapus);
        dealokasi(hapus);
        cout << "Data dengan nomor polisi " << x.nopol << "
berhasil dihapus." << endl;
    } else {
        cout << "Data tidak ditemukan" << endl;
    }
    cout << "\nDATA LIST 1" << endl;
    printInfo(L);

    return 0;
}
```

Screenshoot program



Deskripsi program

Program main.cpp ini adalah program utama yang menjalankan alur interaktif mulai dari meminta input data 4 kendaraan dengan validasi pencegahan nomor polisi kembar, menampilkan isi list, mencari data kendaraan tertentu, menghapus data kendaraan berdasarkan nomor polisi yang dipilih, hingga menampilkan kembali sisa data akhir yang ada di dalam list.

4. **dll.h (guided)**

Source Code

```
#ifndef DLL_H
#define DLL_H

#include <iostream>
#include <string>
using namespace std;

struct Kendaraan {
    string nopol;
    string warna;
    int thnBuat;
};
struct ElmList {
    Kendaraan info;
    ElmList* next;
    ElmList* prev;
};

// STRUKTUR LIST
struct List {
    ElmList* first;
    ElmList* last;
};

// FUNGSI DASAR
void createList(List &L);
ElmList* alokasi(Kendaraan x);
void dealokasi(ElmList* &P);
void printInfo(List L);
ElmList* findElm(List L, string nopol);

// FUNGSI INSERT
void insertFirst(List &L, ElmList* P);
void insertLast(List &L, ElmList* P);
void insertAfter(List &L, ElmList* Prec, ElmList* P);
void insertBefore(List &L, ElmList* Succ, ElmList* P);

// FUNGSI DELETE
void deleteFirst(List &L, ElmList* &P);
void deleteLast(List &L, ElmList* &P);
void deleteAfter(List &L, ElmList* Prec, ElmList* &P);
void deleteBefore(List &L, ElmList* Succ, ElmList* &P);

#endif
```

Deskripsi program

Program dll.h ini adalah berkas header versi lain untuk kerangka dasar Doubly Linked List kendaraan yang menggunakan struct Kendaraan, pointer first dan last berhuruf kecil, serta daftar fungsi operasi penambahan dan penghapusan yang sangat lengkap—mulai dari posisi awal, akhir, setelah, hingga sebelum suatu node acuan.

5. **dll.cpp (guided)**

Source Code

```
#include "dll.h"

// MEMBUAT LIST KOSONG
void createList(List &L) {
    L.first = nullptr;
    L.last = nullptr;
}

// MEMBUAT NODE BARU
ElmList* alokasi(Kendaraan x) {
    ElmList* P = new ElmList;

    P->info = x;
    P->next = nullptr;
    P->prev = nullptr;

    return P;
}

// MEMBEBASKAN MEMORI NODE
void dealokasi(ElmList* &P) {
    if (P != nullptr) {
        delete P;
        P = nullptr;
    }
}

// MENAMPILKAN SEMUA DATA KENDARAAN
void printInfo(List L) {
    ElmList* P = L.first;

    if (P == nullptr) {
        cout << "List kosong." << endl;
        return;
    }

    while (P != nullptr) {
        cout << "Nopol    : " << P->info.nopol << endl;
        cout << "Warna    : " << P->info.warna << endl;
        cout << "Tahun    : " << P->info.thnBuat << endl;
```

```
            cout << "-------------------" << endl;

            P = P->next;
        }
    }

    // MENCARI NODE BERDASARKAN NOMOR POLISI
    ElmList* findElm(List L, string nopol) {
        ElmList* P = L.first;

        while (P != nullptr) {
            if (P->info.nopol == nopol) {
                return P;
            }

            P = P->next;
        }

        return nullptr;
    }

    // INSERT FIRST: MENAMBAHKAN NODE DI AWAL
    void insertFirst(List &L, ElmList* P) {
        if (P == nullptr) {
            return;
        }

        P->prev = nullptr;
        P->next = L.first;

        if (L.first != nullptr) {
            L.first->prev = P;
        } else {
            L.last = P;
        }

        L.first = P;
    }

    // INSERT LAST: MENAMBAHKAN NODE DI AKHIR
    void insertLast(List &L, ElmList* P) {
        if (P == nullptr) {
            return;
        }

        P->next = nullptr;
        P->prev = L.last;
```

```
    if (L.last != nullptr) {
        L.last->next = P;
    } else {
        L.first = P;
    }

    L.last = P;
}

// INSERT AFTER: MENAMBAHKAN NODE SETELAH NODE ACUAN
void insertAfter(List &L, ElmList* Prec, ElmList* P) {
    if (Prec == nullptr || P == nullptr) {
        return;
    }

    if (Prec == L.last) {
        insertLast(L, P);
        return;
    }

    P->next = Prec->next;
    P->prev = Prec;

    Prec->next->prev = P;
    Prec->next = P;
}

// INSERT BEFORE: MENAMBAHKAN NODE SEBELUM NODE ACUAN
void insertBefore(List &L, ElmList* Succ, ElmList* P) {
    if (Succ == nullptr || P == nullptr) {
        return;
    }

    if (Succ == L.first) {
        insertFirst(L, P);
        return;
    }

    P->next = Succ;
    P->prev = Succ->prev;

    Succ->prev->next = P;
    Succ->prev = P;
}

// DELETE FIRST: MENGELUARKAN NODE PERTAMA
void deleteFirst(List &L, ElmList* &P) {
    if (L.first == nullptr) {
```

```
        P = nullptr;
        return;
    }

    P = L.first;

    if (L.first == L.last) {
        L.first = nullptr;
        L.last = nullptr;
    } else {
        L.first = P->next;
        L.first->prev = nullptr;
    }

    P->next = nullptr;
    P->prev = nullptr;
}

// DELETE LAST: MENGELUARKAN NODE TERAKHIR
void deleteLast(List &L, ElmList* &P) {
    if (L.last == nullptr) {
        P = nullptr;
        return;
    }

    P = L.last;

    if (L.first == L.last) {
        L.first = nullptr;
        L.last = nullptr;
    } else {
        L.last = P->prev;
        L.last->next = nullptr;
    }

    P->next = nullptr;
    P->prev = nullptr;
}

// DELETE AFTER: MENGELUARKAN NODE SETELAH NODE ACUAN
void deleteAfter(List &L, ElmList* Prec, ElmList* &P) {
    if (Prec == nullptr || Prec->next == nullptr) {
        P = nullptr;
        return;
    }

    P = Prec->next;
```

```
        if (P == L.last) {
            deleteLast(L, P);
            return;
        }

        Prec->next = P->next;
        P->next->prev = Prec;

        P->next = nullptr;
        P->prev = nullptr;
    }

    // DELETE BEFORE: MENGELUARKAN NODE SEBELUM NODE ACUAN
    void deleteBefore(List &L, ElmList* Succ, ElmList* &P) {
        if (Succ == nullptr || Succ->prev == nullptr) {
            P = nullptr;
            return;
        }

        P = Succ->prev;

        if (P == L.first) {
            deleteFirst(L, P);
            return;
        }

        Succ->prev = P->prev;
        P->prev->next = Succ;

        P->next = nullptr;
        P->prev = nullptr;
    }
```

Deskripsi program

Program dll.cpp ini berisi seluruh rincian kode logika untuk operasi Doubly Linked List kendaraan versi lengkap—mulai dari pembuatan list kosong, alokasi/dealokasi memori, pencetakan data secara berurutan (printInfo), pencarian data (findElm), serta berbagai variasi fungsi penambahan (insertFirst, insertLast, insertAfter, insertBefore) dan penghapusan (deleteFirst, deleteLast, deleteAfter, deleteBefore).

6. **Main.cpp (guided)**

Source Code

```
#include "dll.h"

int main() {
    List L;
    createList(L);

    Kendaraan K1 = {"D001", "Merah", 2022};
    Kendaraan K2 = {"D002", "Hitam", 2023};
    Kendaraan K3 = {"D003", "Biru", 2024};
    Kendaraan K4 = {"D004", "Abu-abu", 2025};
    Kendaraan K0 = {"D000", "Putih", 2021};

    // INSERT LAST: MEMASUKKAN D001 DI AKHIR
    ElmList* P = alokasi(K1);
    insertLast(L, P);

    // INSERT LAST: MEMASUKKAN D002 DI AKHIR
    P = alokasi(K2);
    insertLast(L, P);

    // INSERT FIRST: MEMASUKKAN D000 DI AWAL
    P = alokasi(K0);
    insertFirst(L, P);

    // INSERT AFTER: MEMASUKKAN D003 SETELAH D001
    ElmList* Prec = findElm(L, "D001");

    P = alokasi(K3);
    insertAfter(L, Prec, P);

    // INSERT BEFORE: MEMASUKKAN D004 SEBELUM D002
    ElmList* Succ = findElm(L, "D002");

    P = alokasi(K4);
    insertBefore(L, Succ, P);

    // MENAMPILKAN LIST SETELAH INSERT
    cout << "=== DATA SETELAH INSERT ===" << endl;
    printInfo(L);

    // FIND ELEMENT: MENCARI D001
    P = findElm(L, "D001");

    cout << "=== HASIL PENCARIAN ===" << endl;

    if (P != nullptr) {
```

```
            cout << "Nopol ditemukan: " << P->info.nopol << endl;
            cout << "Warna: " << P->info.warna << endl;
            cout << "Tahun: " << P->info.thnBuat << endl;
        } else {
            cout << "Kendaraan tidak ditemukan." << endl;
        }

        // DELETE AFTER: MENGHAPUS D003 SETELAH D001
        Prec = findElm(L, "D001");

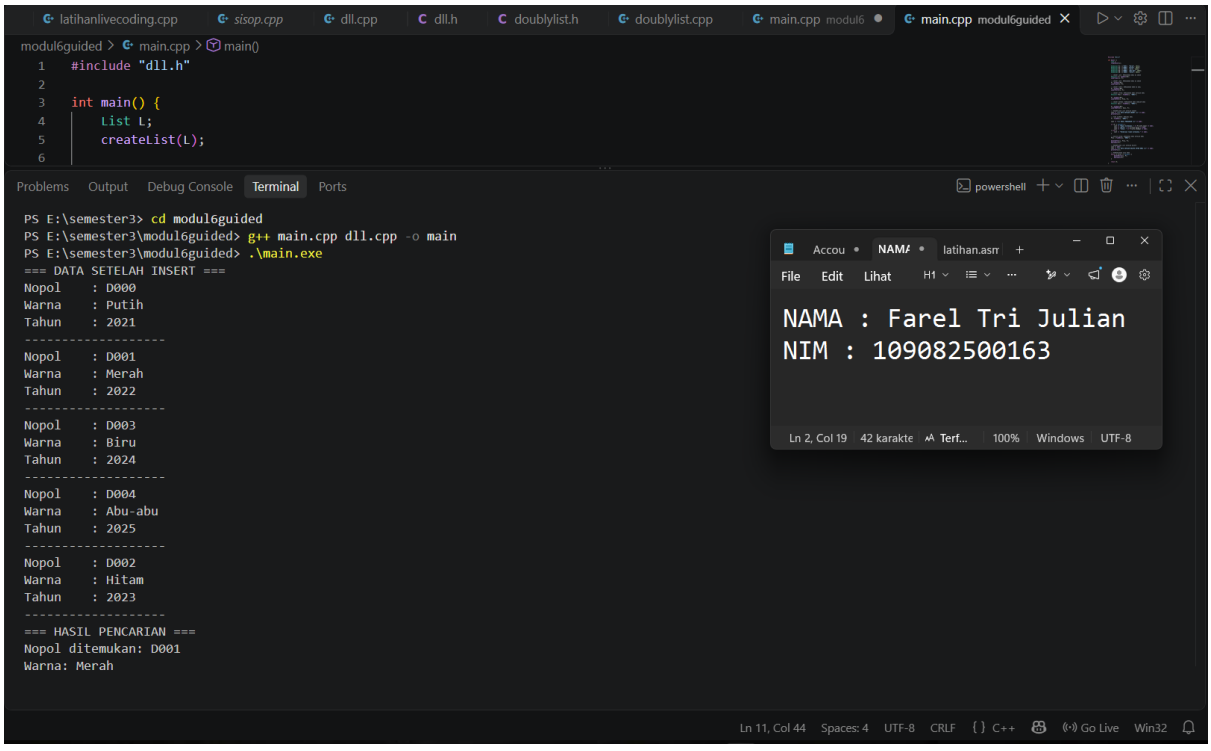
        deleteAfter(L, Prec, P);
        dealokasi(P);

        // MENAMPILKAN LIST SETELAH DELETE
        cout << endl;
        cout << "=== DATA SETELAH DELETE AFTER D001 ===" << endl;
        printInfo(L);

        // MEMBERSIHKAN SISA NODE
        while (L.first != nullptr) {
            deleteFirst(L, P);
            dealokasi(P);
        }

        return 0;
    }
```

Screenshoot program



Deskripsi program

Program main.cpp ini adalah program utama untuk menguji seluruh fungsi operasi Doubly Linked List kendaraan mulai dari memasukkan data menggunakan insertLast, insertFirst, insertAfter, dan insertBefore, menampilkan data, mencari nomor polisi tertentu, menghapus node menggunakan deleteAfter, hingga membersihkan seluruh sisa memori list di akhir program.